

Introduction to Data Analysis with Python

Spring Semester 2026

Python basics

Cecilia Graiff
ALMAAnaCH, Inria Paris

Acknowledgements

This handbook is a work in progress created for the lectures in the course "Advanced Data Analysis with Python", given at SciencesPo in fall 2025. It is not a final or distributed version and it is susceptible to changes.

To write this handbook, I relied on several sources, which I will list below. This list is also being updated. Please note that the version you are using might not contain all references. In addition to this list, I leveraged the **official online documentation** of the Python packages that are handled.

- [LxMLS lab guide](#)
- [Python Data Science Handbook](#)
- [An Introduction to Statistical Learning](#)
- [Introduction to Applied Linear Algebra](#)
- [Linear Algebra with Python](#)

Contents

1	Data structures	3
1.1	Tuple	5
1.2	Dictionary	6
1.3	Set	7

1 Data structures

List

A **list** is a mutable, ordered collection in Python, defined using square brackets []. Lists can contain elements of **any data type** and can contain **duplicates**. Example of basic operations on lists are:

Creating a List

```
numbers = [1, 2, 3, 4, 5]
```

Indexing

Indexing refers to the process of accessing a specific element in a sequence, in this case a list, using its position (called **index number**). **Indexing in Python starts at 0.**

It is very important to remember that **indexing in Python starts at 0**, meaning that the first element in a sequence has an index of 0, the second element has an index of 1, and so on. Thus, if you want to access the n^{th} element of your list, you will find it at position $n-1$.

Below is an example of list indexing.

```
my_list = [1, 2, 3, 4, 5]

print(my_list[0])    # 1 (first element)
print(my_list[-1])   # 5 (last element)
print(my_list[4])     # 5 (4th element, starting indexing at 0)
```

Slicing

Slicing is the operation of extracting some elements from a list based on their indexes. It allows selecting a range of elements from a list by creating a shallow copy of a list containing only those elements.

Extract a portion of the list:

```
print(my_list[1:4])    # [2, 3, 4]
print(my_list[:3])     # [1, 2, 3]
print(my_list[::2])    # [1, 3, 5] (every 2nd element)
```

Appending

Appending is the operation of adding an element to a list. It is performed with the `append` command.

```
my_list.append(4)

print(my_list)  # [1, 2, 3, 4, 5, 4]
```

Removing

```
my_list.remove(4) # Removes element 4, not element number 4!
```

Combining Lists

```
a = [1, 2]
b = [3, 4]
combined = a + b          # [1, 2, 3, 4]
```

Looping Through a List

```
for elem in my_list:
    print(elem)
```

We will see that NumPy offers a valid alternative to looping.

1.1 Tuple

A **tuple** in Python is an immutable, ordered collection of elements. Tuples can hold elements of different data types and are defined using parentheses (). Keep in mind that tuples are **immutable** and their contents cannot be changed after creation. Some examples of basic operations on tuples:

Creating a tuple

```
my_tuple = (1, 2, 3, 4)
```

Indexing and Slicing

```
print(my_tuple[0])      # 1
print(my_tuple[-1])     # 4
print(my_tuple[1:3])    # (2, 3)
```

Length of a tuple

```
print(len(my_tuple))    # 4
```

Looping through a tuple

```
for item in my_tuple:
    print(item)
```

Nested tuple and unpacking

```
nested = (1, (2, 3))
a, b = (10, 20)
print(nested[1][0])      # 2
print(a, b)              # 10 20
```

1.2 Dictionary

A **dictionary** (`dict`) is an unordered collection of key-value pairs in Python. Dictionaries are defined using curly braces `{}`. Each key is associated to a specific value. Keys must be unique and immutable.

Creating a Dictionary

```
person = {  
    "name": "Alice",  
    "age": 25,  
    "job": "student"  
}
```

Accessing Values

```
print(person["name"])    # Alice  
print(person.get("age")) # 25
```

Adding/Updating Values

```
person["email"] = "alice@example.com"  
person["age"] = 26
```

Deleting Items

```
del person["email"]
```

Looping Through a Dictionary

```
for key, value in person.items():  
    print(key, value)
```

1.3 Set

A **set** is an unordered collection with no duplicate elements. **Curly braces** or the `set()` function can be used to create sets. Sets can be used also to **remove duplicates from a list**.

Attention! When creating an **empty set**, **you have to use `set()`**, **not curly braces**; otherwise, **you will create an empty dictionary**!

Creating a Set

```
fruits = {"apple", "banana", "cherry"}
my_set = set() # Creates an empty set called my_set
```

Adding Elements

```
fruits.add("orange")
```

Removing Elements

```
fruits.remove("banana") # Error if not found
fruits.discard("grape") # No error if not found
```

Set Operations

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a | b) # Union: {1, 2, 3, 4, 5}
print(a & b) # Intersection: {3}
# you can also use .intersection
print(a - b) # Difference: {1, 2}
print(a ^ b) # Symmetric difference: {1, 2, 4, 5}
# you can also use .symmetric_difference
```

Looping Through a Set

```
for fruit in fruits:  
    print(fruit)
```